

-
- Course Code: PA1489
 - Name: Basic Software Engineering
-

GenAI Policy

Note: Throughout this document, “you” refers to students enrolled in the course, “we” refers to examiners/supervisors/teaching team.

The goal of the course assignments is to demonstrate your ability to make use of basic software engineering tools that are an integral part of a modern software development environment. This includes your editor, compilers, debugger, automated test tools, programming language specific documentation tools, microservice container tools, etc.

All GenAI use must align with the objectives and intended learning outcomes defined in the course syllabus. GenAI should support, rather than replace, your demonstration of competencies such as critical thinking and scientific writing.

Sources

This document is partially based on the GenAI policy template by Franz Zieris:

GenAI Policy Template, Franz Zieris, Blekinge Institute of Technology, 2026. Available at: <https://github.com/bth-dipt-teaching/genai-policy/blob/main/template.md> Licensed under CC BY 4.0.

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC 2119] (<https://www.rfc-editor.org/info/rfc2119/>).

The document further makes use of the AI Assessment Scale (AIAS) developed by Mike Perkins, Leon Furze, Jasper Roe, and Jason MacVaugh in 2023/2024.

General Principles

Full Responsibility and Accountability

- You **MUST** take total ownership of every word, figure, and line of code; verify all AI-assisted output against primary sources.

- You **MUST NOT** assume that the AI knows best (e.g., using “The AI said so” as a justification), that details do not matter, or that examiners will not notice.
- You **SHOULD** make sure you understand the output of your AI tool and then write/use your own interpretation of it.
- **Motivation:** You are effectively “signing” your assignments as your intellectual output. You must be able to explain, justify, and trace the origin of any part of it. In group work, accountability is collective; all authors are responsible for the entire content.
- **Example:** A student experimented with ChatGPT for generating some program code for a task, but then re-typed it by themselves on a level where they understand what the code does.

Do not Plagiarise

- **Comment:** This is a wider question than just the use of GenAI, but it applies equally to material produced by GenAI tools.
- You **MUST** properly cite and credit any material used in your assignments. This includes the *work* of others, and this includes output from GenAI tools.
- You **MUST NOT** copy-paste or otherwise present the work of others as if it was your own.
- You **SHOULD** make it a habit to always summarise text that you want to use and to immediately add a reference to where you got it from.
- You **MAY** include the prompts you used to generate code or text from a GenAI tool, in order to discuss it with your teacher.
- **Motivation 1:** The assignments are an opportunity for you to show what *you* have learned. By including sources and/or prompts we are able to help you learn more.
- **Motivation 2:** Plagiarism is a serious academic offence: You *must always* give credit to the original author(s).
- **Example:** A student asks Claude to generate different suggestions for the code for a particular task, but then writes their own version which is a combination of several suggestions. They add the prompts used in a source code comment and discuss these with the teacher.
- **Example:** A student finds several ready-made solutions to a programming task online, and writes their own solution inspired by what they have read. They include links to the used pages in a source code comment.

Use AI Tools to Explain, not to Do.

- You **MAY** use AI to explain different commands and overall workflow.
- You **MAY** see this as an opportunity to practice your ability to find and read documentation and instruction manuals instead of relying on AI tools.
- You **SHOULD** make sure that you understand the tools that you use.
- You **SHOULD** verify what the AI tells you through independent tests.

- You **SHOULD NOT** rely on that the GenAI understand the tools for you.
- You **MUST NOT** blame the AI for any mistakes it does. Those mistakes are yours to catch.
- **Motivation:** You must at least be able to identify when GenAI tools suggests something wrong. Your value as an engineer is further increased if you are also able to work independently with minimal tool or AI support.
- **Example:** An engineer asked an AI tool to generate code to implement a feature and realised that the generated code would break the running system and cost millions of dollars. If they had not independently reviewed the generated code before committing they would not have caught this.
- **Comment:** In industry, the line between explaining and doing is blurred, but **SHOULD** only be crossed by engineers *who already know what they are doing*. You are still students and are still learning the trade.

Specific Guidelines

Installing Software

- You **MAY** use AI to assist with installing the required software.
- You **SHOULD** make sure you have a package manager installed that can automate most software installations, for the future.
- **Motivation:** This is not an examined part of the course and you should preferably not get stuck on this point so you are free to do as you please. In order to gain more control and understanding, it is better if you use a dedicated package manager.
- **Example:** A student is struggling to get git working in WSL2, so they ask Copilot for help, and follows the instructions received.

Working with Configuration Management

- You **MUST** make sure that you learn how to use configuration management tools and the thereby associated commands without the help of AI tools.
- You **MUST** be able to demonstrate the use of these tools and commands to a teacher.
- You **MUST NOT** allow AI tools to commit work for you.
- You **MAY** use AI to assist with specific tasks such as drafting text and code for the commit, as long as this does not conflict with any other guidelines.
- **Motivation:** Configuration management is a core skill for a software engineer and something which you will be using on a daily basis. When you make a commit, you are effectively “signing” your work and your intellectual output. You must be able to explain, justify, and trace the origin of any part of it.
- **Example:** A student use a GenAI tool to explain how to stage work before

committing, but then types the commands on their own in a separate terminal window.

Working with Containers

- You **MAY** be inspired by existing Dockerfiles and Docker compose files that you find online.
- You **MAY** use GenAI tools to generate a Dockerfile or Docker compose file for you, that you then interpret and re-type in your own project. Focus on directing AI to achieve your goals while demonstrating your critical thinking.
- You **MUST** ensure that your Dockerfiles and Docker compose files *only* do what is necessary.
- You **MUST** ensure that any edited Dockerfile or Docker compose file is *only* changed where you expect it to be.
- You **MUST** be able to read and explain a Dockerfile or a Docker compose file.
- **Motivation:** AI tools regularly generate too much code or too complex solutions, whereas manually written examples are often shorter and more to the point. Reading many examples of existing Dockerfiles and Docker compose files, and using them as inspiration for your own projects means that you learn how to see through the AI's tendency to over-complicate and bloat so that you are instead able to understand and focus on the core principles.
- **Example:** A student asks Claude Code for help to write the remaining service specifications in a partially completed Docker compose file, but writes the specification for environment variables and dependencies on other services manually. They then compare the new file with the original one and makes sure that there are no other accidental changes.
- **Comment:** Because of the tendency to bloat and over-complicate, AI generated software code is actually really bad for learning. It is much easier to read and understand a minimal example generated by an engineer.

Testing and Debugging

- You **MUST NOT** use GenAI tools to write the automated tests or to help debug the given program.
- You **MAY** copy from the already existing unit tests and modify them.
- You **MAY** use GenAI to explain the existing unit tests for you.
- **Motivation:** The learning objective of this assignment is not to find the bug, but to practice the steps you may take in order to identify what the bug is, where it may be hiding, how to fix it, and how to add this test to the suite of unit tests.
- **Example:** A student heard what the bug consists of from another student, but then wrote their own unit test and used the debugger to independently find it and fix it.

Implementation

- You **MUST NOT** use GenAI tools to write any source code or code documentation.
- You **MAY** read and be inspired by the existing code in the project, and then modify it to your needs.
- You **MAY** use GenAI to explain the existing code for you.
- You **SHOULD** discuss with your friends.
- **Motivation:** Any software code required in this course is mostly trivial, the intended learning outcome is everything around it, e.g. where to write the code, how to call other methods, how to write clear and understandable code, what to name functions and variables, etc. Discussing with your friends is a vital learning tool, and the big difference between that and using an AI tool is that your friends know when to stop explaining and let you find the solution on your own. The code documentation represent your interpretation of what a function does, which we would like to use to help you develop your code-understanding skills.
- **Example:** A student is struggling with how to write the source code for an assignment, so they discuss with their classmates who show and explain how they solved the same task.

Engineering Diary

- You **MUST NOT** use GenAI tools to write your engineering diary.
- You **MAY** use GenAI tools to assist you in formatting the document, e.g. explaining how to format a text in Markdown.
- **Motivation:** The diary is your personal notebook and your way of keeping a record of what you know or have yet to learn. Using an AI tool to generate this diary entirely defeats the purpose of it. In this course we do some basic sanity checks of the diary *just* to make sure that you have gotten started in establishing a healthy habit of journalling your work and your learning. Finding and writing in your own voice is important for you to later understand your notes. When marking, we do not look for well formed sentences or completeness.
- **Example:** One student submitted scans of hand-written pages from a notebook. Another student used a dedicated tool, but most students just use a simple textfile.